

PRD.md — Synth NET | To-Do List

Project Overview

Project Name

Synth NET | To-Do List

Objective

Membangun aplikasi To-Do List modern berbasis React + Supabase untuk pembelajaran fullstack development menggunakan AI-assisted workflow.

Project ini harus menggunakan modular monolith architecture dan scalable untuk future development.

AI Agent bertugas melakukan:

- initial project setup
 - dependency installation
 - folder scaffolding
 - frontend setup
 - Supabase integration
 - authentication setup
 - CRUD todo setup
 - clean architecture implementation
-

Tech Stack

Frontend

- Bun
- React
- Vite
- TypeScript
- React Router DOM
- Zustand
- TanStack Query
- React Hook Form
- Zod
- Sonner

- Lucide React

UI & Design

Use:

- getdesign.md

Command:

```
npx getdesign@latest add expo
```

AI Agent should use generated design system components from getdesign.md as base UI components.

Backend

Use:

- Supabase

Supabase project already exists:

```
Synth-NET-ToDo-List
```

Backend services:

- Supabase Auth
 - Supabase PostgreSQL
 - Row Level Security (RLS)
-

Architecture

Use:

```
Modular Monolith Architecture
```

Project must be structured for future scalability.

Future scalability targets:

- Redis
- Audit Trails
- Notification system
- Realtime
- Queue workers
- Background jobs

Initial Project Setup

AI Agent must:

1. Create project

Use:

```
bun create vite
```

Requirements:

- React
- TypeScript

2. Install dependencies

Install:

```
bun add @supabase/supabase-js
bun add react-router-dom
bun add zustand
bun add @tanstack/react-query
bun add react-hook-form
bun add zod
bun add sonner
bun add lucide-react
```

Dev dependencies:

```
bun add -d @types/node
```

3. Setup getdesign.md

Run:

```
npx getdesign@latest add expo
```

4. Setup ESLint & Clean Formatting

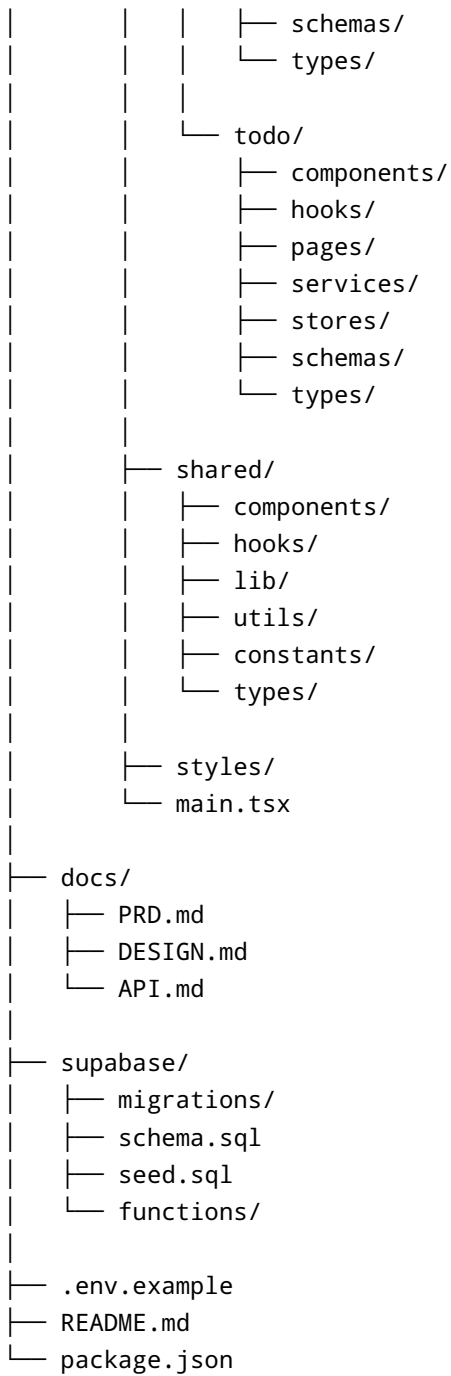
AI Agent should configure:

- clean imports
- strict TypeScript
- readable folder structure
- scalable naming convention

Folder Structure

AI Agent must create this structure:

```
synth-net-todo/
|
├── apps/
│   └── web/
│       ├── src/
│       ├── app/
│       │   ├── providers/
│       │   ├── router/
│       │   └── layouts/
│       └── modules/
│           ├── auth/
│           │   ├── components/
│           │   ├── hooks/
│           │   ├── pages/
│           └── services/
```



AI Agent must create:

```
VITE_SUPABASE_URL=  
VITE_SUPABASE_ANON_KEY=
```

inside:

```
.env.example
```

Supabase Integration

AI Agent must:

Create Supabase client

Location:

```
shared/lib/supabase.ts
```

Implementation requirements:

- typed client
- reusable singleton pattern
- environment variable validation

Database Schema

AI Agent must create SQL schema files.

profiles table

```
create table profiles (  
  id uuid primary key references auth.users(id) on delete cascade,  
  full_name text,  
  avatar_url text,  
  created_at timestamptz default now()  
);
```

todos table

```
create table todos (  
  id uuid primary key default gen_random_uuid(),  
  
  user_id uuid not null  
    references auth.users(id)  
    on delete cascade,  
  
  title text not null,  
  description text,  
  
  is_completed boolean default false,  
  
  created_at timestamptz default now(),  
  updated_at timestamptz default now()  
);
```

Row Level Security (RLS)

AI Agent must enable RLS.

```
alter table todos enable row level security;
```

Policy:

```
create policy "Users can manage own todos"  
on todos  
for all  
using (auth.uid() = user_id)  
with check (auth.uid() = user_id);
```

Authentication Requirements

AI Agent must implement:

Authentication Features

- Sign Up
 - Sign In
 - Sign Out
 - Google OAuth
 - Email & Password Authentication
-

Todo Features

AI Agent must implement:

CRUD Todo

- Create todo
 - Read todo list
 - Update todo
 - Delete todo
 - Mark todo as completed
-

Routing

AI Agent should implement:

Public Routes

- /login
- /register

Protected Routes

- /dashboard

Use protected route pattern.

State Management

Use:

- Zustand

- TanStack Query

Requirements:

- clean async handling
 - reusable hooks
 - modular services
-

Coding Standards

AI Agent must follow:

General Rules

- strict TypeScript
 - reusable components
 - no duplicated logic
 - modular architecture
 - feature-based folder structure
 - clean naming convention
-

Naming Convention

Files

```
todo.service.ts  
todo.types.ts  
todo.schema.ts  
todo.hook.ts
```

Components

```
TodoCard.tsx  
TodoForm.tsx  
TodoList.tsx
```

UI Requirements

Use modern minimal design.

Requirements:

- responsive
 - dark mode ready
 - clean spacing
 - mobile friendly
 - smooth interaction
-

Future Scalability

Codebase should be prepared for future addition:

- Redis
- Notification system
- Realtime sync
- Audit trail logging
- Queue workers
- API abstraction layer
- Backend service extraction

Avoid tightly coupled code.

README Requirements

AI Agent must generate README.md containing:

- project overview
 - setup instructions
 - environment setup
 - Supabase setup
 - available scripts
 - architecture explanation
-

Final AI Agent Goal

At the end of initial setup:

AI Agent should produce:

- running frontend app
- connected Supabase client

- authentication flow
- todo CRUD flow
- modular architecture
- clean scalable foundation

Application should run successfully using:

```
bun run dev
```